Lab 1 – Projekt algorytmu równoległego metodą PCAM

Zadanie 3: ugięcie równomiernie obciążonej membrany kwadratowej (równanie Poissona)

Antoni Smółka

1. Problem

Membrana w kształcie kwadratu o boku a, brzeg sztywno zamocowany, obciążona równomiernym ciśnieniem $p > 0$, napięcie T. Szukamy ugięcia $h(x, y)$ spełniającego

$$\frac{\partial^2 h}{\partial x^2} + \frac{\partial^2 h}{\partial y^2} = -\frac{p}{T}, \qquad (x, y) \in \Omega = (0, a) \times (0, a),$$

z warunkiem brzegowym Dirichleta $h = 0$ na $\partial\Omega$ (brzeg nieodkształcony).

2. Algorytm sekwencyjny

2.1 Dyskretyzacja (metoda różnicowa)

Wprowadzamy jednorodną siatkę $(N+1) \times (N+1)$ węzłów z krokiem $\Delta = a/N$: $x_i = i\Delta,\ y_j = j\Delta,\ i, j = 0, \dots, N$, oraz $h_{i,j} \approx h(x_i, y_j)$. Węzły brzegowe ($i \in \{0, N\}$ lub $j \in \{0, N\}$) mają znaną wartość $h_{i,j} = 0$; niewiadomych jest $(N-1)^2$ (węzły wewnętrzne).

Z rozwinięcia Taylora w punkcie x_i:

$$h(x_i \pm \Delta, y_j) = h \pm \Delta h_x + \tfrac{\Delta^2}{2} h_{xx} \pm \tfrac{\Delta^3}{6} h_{xxx} + O(\Delta^4)$$

po dodaniu obu rozwinięć wyrazy nieparzyste się redukują, więc otrzymujemy różnicę centralną drugiego rzędu:

$$\frac{\partial^2 h}{\partial x^2}\Big|_{i,j} = \frac{h_{i+1,j} - 2h_{i,j} + h_{i-1,j}}{\Delta^2} + O(\Delta^2), \qquad \frac{\partial^2 h}{\partial y^2}\Big|_{i,j} = \frac{h_{i,j+1} - 2h_{i,j} + h_{i,j-1}}{\Delta^2} + O(\Delta^2).$$

Podstawiając do równania otrzymujemy **pięciopunktowy schemat różnicowy** dla każdego węzła wewnętrznego:

$$\frac{h_{i+1,j} + h_{i-1,j} + h_{i,j+1} + h_{i,j-1} - 4h_{i,j}}{\Delta^2} = -\frac{p}{T}, \qquad i, j = 1, \dots, N-1.$$

2.2 Układ równań liniowych

Numerując niewiadome wierszami, $m = (j-1)(N-1) + i$, dostajemy układ $A\mathbf{h} = \mathbf{b}$ rozmiaru $n = (N-1)^2$, gdzie

- A jest macierzą rzadką, pięciodiagonalną (blokowo trójdiagonalną): na diagonali 4, przy sąsiadach -1,
- $b_m = \Delta^2 p/T$ (stała – obciążenie jest równomierne; wkład brzegu jest zerowy, bo $h = 0$ na $\partial\Omega$).

Macierz jest symetryczna, dodatnio określona i słabo diagonalnie dominująca (nieprzywiedlna), więc metody iteracyjne Jacobiego i Gaussa–Seidla są zbieżne. Rozwiązanie metodą bezpośrednią (eliminacja Gaussa) jest nieopłacalne: wypełnienie macierzy niszczy rzadkość ($O(n^2)$ pamięci w ogólnym przypadku), a każdy wiersz ma tylko 5 niezerowych elementów.

2.3 Wyprowadzenie wzoru roboczego (metoda Jacobiego)

Rozkładamy $A = D + L + U$ (D – diagonala, L, U – część pod- i naddiagonalna). Z $(D + L + U)\mathbf{h} = \mathbf{b}$:

$$D\mathbf{h} = \mathbf{b} - (L+U)\mathbf{h} \quad \Longrightarrow \quad \mathbf{h}^{(k+1)} = D^{-1}\big(\mathbf{b} - (L+U)\mathbf{h}^{(k)}\big).$$

Ponieważ $D = 4I$, a $(L + U)$ ma w wierszu m wartości -1 w kolumnach odpowiadających 4 sąsiadom, w zapisie siatkowym otrzymujemy **wzór roboczy**:

$$\boxed{h_{i,j}^{(k+1)} = \frac{1}{4}\left(h_{i+1,j}^{(k)} + h_{i-1,j}^{(k)} + h_{i,j+1}^{(k)} + h_{i,j-1}^{(k)} + \Delta^2 \frac{p}{T}\right), \qquad i,j = 1,\dots,N-1}$$

czyli nowa wartość w węźle to średnia z czterech sąsiadów z poprzedniej iteracji plus człon źródłowy. Kluczowe dla równoległości: **każda nowa wartość zależy wyłącznie od wartości z poprzedniej iteracji**, więc w obrębie jednej iteracji wszystkie węzły można liczyć niezależnie.

(Gauss–Seidel, $h_{i,j}^{(k+1)} = \frac{1}{4}(h_{i+1,j}^{(k)} + h_{i-1,j}^{(k+1)} + h_{i,j+1}^{(k)} + h_{i,j-1}^{(k+1)} + \Delta^2 p/T)$, zbiega ok. 2 razy szybciej, ale wprowadza zależność od wartości już policzonych w bieżącej iteracji – sekwencyjny przebieg „po fali". Dlatego jako bazę do zrównoleglenia wybieram Jacobiego; wariant red-black Gaussa–Seidla omawiam w fazie C.)

2.4 Warunek stopu

$$\max_{i,j} \left|h_{i,j}^{(k+1)} - h_{i,j}^{(k)}\right| < \varepsilon \quad \text{(lub limit iteracji } k_{\max}\text{)}.$$

2.5 Pseudokod

```
Δ  ← a / N
f  ← Δ² · p / T
h_old[0..N][0..N] ← 0          // brzeg = 0 na stałe, wnętrze: przybliżenie startowe 0
h_new ← h_old
repeat
    diff ← 0
    for i ← 1 .. N-1
        for j ← 1 .. N-1
            h_new[i][j] ← 0.25 · (h_old[i+1][j] + h_old[i-1][j]
                                  + h_old[i][j+1] + h_old[i][j-1] + f)
            diff ← max(diff, |h_new[i][j] - h_old[i][j]|)
    swap(h_old, h_new)            // podwójne buforowanie – Jacobi wymaga starej kopii
until diff < ε  or  k ≥ k_max
```

2.6 Koszt

- jedna iteracja: $\Theta(N^2)$ (ok. 5 operacji zmiennoprzecinkowych na węzeł), pamięć $2(N+1)^2$,
- promień spektralny macierzy iteracji Jacobiego: $\rho = \cos(\pi/N) \approx 1 - \frac{\pi^2}{2N^2}$, więc liczba iteracji potrzebnych do zredukowania błędu o stały czynnik to $\Theta(N^2)$,
- łącznie $\Theta(N^4)$ – dla dużych siatek obliczenia są kosztowne, co uzasadnia zrównoleglenie.

Weryfikacja: dla $N = 20$ algorytm zbiega w ok. 1100 iteracjach ($\varepsilon = 10^{-9}$) do ugięcia w środku $h_{\max} \approx 0{,}0735\, pa^2/T$, co zgadza się z wartością analityczną $\approx 0{,}0737\, pa^2/T$ (rozwiązanie szeregiem Fouriera).

3. Projekt algorytmu równoległego – PCAM

3.1 P – Partycjonowanie (Partitioning)

Dekompozycja domenowa. Dane problemu to siatka węzłów, a obliczenia (wzór roboczy) są identyczne w każdym węźle wewnętrznym. Naturalna jest więc dekompozycja danych: **najdrobniejsze zadanie = jeden węzeł wewnętrzny** (i, j), który przechowuje $h_{i,j}$ i w każdej iteracji liczy $h_{i,j}^{(k+1)}$ ze wzoru roboczego oraz lokalną różnicę $|h_{i,j}^{(k+1)} - h_{i,j}^{(k)}|$.

- Liczba zadań: $(N-1)^2$ – o rzędy wielkości więcej niż procesorów, więc dekompozycja daje dużą elastyczność dalszych faz i skaluje się z rozmiarem problemu.
- Zadania są **jednorodne**: każde wykonuje tę samą liczbę operacji (stałe p/T, jednorodny kwadrat bez otworów) – dobry punkt wyjścia do równomiernego obciążenia.
- Węzły brzegowe nie są zadaniami – ich wartość $h = 0$ jest stała i znana z góry, nie wymaga obliczeń.
- Dekompozycja funkcjonalna nie ma tu sensu – w jednej iteracji jest tylko jeden rodzaj obliczeń. Dodatkowo wyodrębniamy jedynie logiczne zadanie **sprawdzenia warunku stopu** (redukcja `max` po wszystkich węzłach).
- Równoległość jest w obrębie iteracji (wszystkie węzły naraz); kolejne iteracje są od siebie zależne, więc wymiar „czasu iteracyjnego" pozostaje sekwencyjny.

3.2 C – Komunikacja (Communication)

Komunikacja lokalna (stencil). Z wzoru roboczego: zadanie (i, j) potrzebuje w iteracji $k+1$ wartości $h^{(k)}$ od czterech sąsiadów $(i \pm 1, j), (i, j \pm 1)$. Zatem w każdej iteracji każde zadanie:

1. wysyła swoją wartość $h_{i,j}^{(k)}$ do 4 sąsiadów,
2. odbiera 4 wartości od sąsiadów,
3. liczy $h_{i,j}^{(k+1)}$.

Charakterystyka: komunikacja **lokalna, strukturalna** (siatka 2D), **statyczna** (te same pary w każdej iteracji) i **synchroniczna** (iteracja $k+1$ może ruszyć dopiero, gdy są wszystkie wartości z k). Zadania przy brzegu mają mniej partnerów – sąsiad brzegowy to stała 0 znana lokalnie, bez komunikacji.

```
          (i,j+1)
             |
(i-1,j) - (i,j) - (i+1,j)        5-punktowy szablon: 4 wiadomości
             |                    wejściowe i 4 wyjściowe na iterację
          (i,j-1)
```

Komunikacja globalna – warunek stopu. Decyzja o zakończeniu wymaga max z lokalnych różnic ze wszystkich zadań, a następnie rozgłoszenia wyniku (all-reduce). Naiwnie (jedno zadanie zbiera wszystko) to $O(N^2)$ wiadomości do jednego punktu – wąskie gardło. Rozwiązanie: **redukcja drzewiasta** (motyl/drzewo binarne) w $O(\log)$ krokach. Ponieważ jest to jedyna operacja globalna, a jej koszt rośnie z liczbą procesów, warunek stopu warto sprawdzać **co** m **iteracji** (np. co 10–50) – kosztem co najwyżej m zbędnych iteracji.

Uwaga o Gaussie–Seidlu. Gdyby użyć klasycznego G-S, zadanie (i, j) czekałoby na nowe wartości od $(i-1, j)$ i $(i, j-1)$ – powstaje fala obliczeń po przekątnych i mała równoległość. Wariant **red-black** (szachownica) rozwiązuje to: najpierw wszystkie „czerwone" węzły ($i + j$ parzyste) – zależą tylko od czarnych – potem wszystkie „czarne". Daje to dwie fazy komunikacji na iterację, ale ok. 2× szybszą zbieżność i możliwość nadrelaksacji (SOR). Podstawowy projekt pozostaje przy Jacobim.

3.3 A – Aglomeracja (Agglomeration)

Zadanie „jeden węzeł” jest zdecydowanie za drobne: ok. 5 flopów obliczeń na 4 wysłane i 4 odebrane wiadomości. Koszt opóźnienia (latency) wiadomości dominowałby nad obliczeniami. Łączymy więc węzły w większe zadania, wykorzystując efekt **powierzchnia/objętość**: obliczenia rosną z polem bloku, komunikacja tylko z jego obwodem (wymieniamy wyłącznie wartości na krawędziach bloku).

Rozważam dwie aglomeracje dla P zadań:

	paski (1D) – N/P wierszy	bloki (2D) – $\frac{N}{\sqrt{P}} \times \frac{N}{\sqrt{P}}$
obliczenia na zadanie	N^2/P	N^2/P
liczba wiadomości / iterację	2	4
objętość komunikacji / iterację	$2N$	$4N/\sqrt{P}$
czas komunikacji	$2(t_s + Nt_w)$	$4(t_s + \frac{N}{\sqrt{P}}t_w)$

(t_s – czas startu wiadomości, t_w – czas przesłania jednej wartości.)

- Dla pasków objętość komunikacji **nie maleje** ze wzrostem P, więc stosunek komunikacji do obliczeń rośnie liniowo z P – słaba skalowalność.
- Bloki 2D mają mniejszą objętość już dla $P > 4$ i skalują się lepiej; paski mają mniej wiadomości (lepsze przy dużym t_s i małej liczbie procesorów, prostsza implementacja – ciągłe wiersze w pamięci).
- **Wybór: bloki 2D** (dla małego P, np. jeden węzeł obliczeniowy z kilkoma rdzeniami, paski są akceptowalne).

Szczegóły aglomeracji:

- Każdy blok przechowuje swoje wartości oraz **warstwę widmową (ghost/halo) szerokości 1** wokół bloku. Na początku iteracji wymienia z 4 sąsiednimi blokami krawędzie (wiersze/kolumny po $N/\sqrt{P}$ wartości) **jedną zbiorczą wiadomością** na kierunek – zamiast $N/\sqrt{P}$ pojedynczych.
- Bloki przy brzegu membrany mają halo wypełnione stałymi zerami – brak komunikacji w tych kierunkach.
- Wnętrze bloku można liczyć **w trakcie** wymiany halo (komunikacja nieblokująca), a dopiero potem krawędź bloku – nakładanie komunikacji i obliczeń.
- Warunek stopu: każdy blok najpierw liczy **lokalne** maksimum różnic (bez komunikacji), w redukcji globalnej uczestniczy tylko P wartości.
- Opcjonalnie: halo szerokości $w > 1$ pozwala wymieniać dane raz na w iteracji kosztem redundantnych obliczeń – opłaca się przy dużym t_s.

3.4 M – Mapowanie (Mapping)

- Liczba zadań po aglomeracji = liczba procesorów P: **jeden blok na procesor**. Procesy tworzą wirtualną topologię kartezjańską $\sqrt{P} \times \sqrt{P}$ (np. `MPI_Cart_create`), a blok (I, J) trafia do procesu o współrzędnych (I, J) – sąsiedzi w siatce są sąsiadami w topologii, co sprzyja umieszczaniu ich na tym samym węźle/blisko w sieci.
- **Mapowanie statyczne** wystarcza: obciążenie jest jednorodne (kwadrat, stała prawa strona, stały koszt na węzeł, ta sama liczba iteracji wszędzie), więc nie jest potrzebne dynamiczne równoważenie obciążenia. Gdy $N - 1$ nie dzieli się przez $\sqrt{P}$, rozmiary bloków różnią się co najwyżej o 1 wiersz/kolumnę.
- Lekka nierównowaga: bloki brzegowe/narożne komunikują się z 2–3 sąsiadami zamiast 4 – pomijalne.
- Gdy P nie jest kwadratem, używamy siatki $P_x \times P_y$ możliwie bliskiej kwadratu (np. `MPI_Dims_create`).
- Na pamięci współdzielonej (OpenMP): ta sama dekompozycja jako statyczny podział wierszy/bloków między wątki (`schedule(static)`), wymiana halo znika (wspólna tablica), zostaje bariera po każdej iteracji i redukcja `max`.

Szacowany czas jednej iteracji (bloki 2D, sprawdzanie stopu co m iteracji, t_c – czas obliczeń na węzeł):

$$T_{\text{iter}} \approx t_c \frac{N^2}{P} + 4\Big(t_s + \frac{N}{\sqrt{P}} t_w\Big) + \frac{1}{m}\, 2 \log_2 P\, (t_s + t_w)$$

Pierwszy człon maleje jak $1/P$, komunikacja tylko jak $1/\sqrt{P}$ – algorytm jest efektywny, dopóki bloki są duże ($N/\sqrt{P} \gg 1$), tj. przy skalowaniu N razem z P (izoefektywność: $N^2 \sim P$).

4. Podsumowanie

faza	decyzja dla zadania membrany
P	dekompozycja domenowa: 1 zadanie = 1 węzeł wewnętrzny siatki, $(N-1)^2$ zadań, brzeg $h = 0$ stały
C	lokalna, statyczna wymiana z 4 sąsiadami (5-punktowy szablon) + globalna redukcja max dla warunku stopu
A	bloki 2D $\frac{N}{\sqrt{P}} \times \frac{N}{\sqrt{P}}$ z warstwą halo, zbiorcze wiadomości, lokalne maksimum przed redukcją
M	jeden blok na procesor, topologia kartezjańska $\sqrt{P} \times \sqrt{P}$, mapowanie statyczne